

配列に型を付ける(実践篇)

Practice to type array



pixiv Inc.
USAMI Kenta

pixiv

お前誰よ



- うさみけんた (@tadsan) / Zonu.EXE / にゃんだーすわん
- ピクシブ株式会社 pixiv事業本部 エンジニア
 - 最近はピクシブ百科事典(dic.pixiv.net)を開発しています (PSR-15!)
- Emacs Lisper, PHPer
 - Emacs PHP Modeを開発しています (2017年-)
- PHPカンファレンス実行委員、PHPerKaigi コアスタッフ

tadsanのあれこれが読める場所



- <https://tadsan.fanbox.cc/>
- <https://scrapbox.io/php/>
- <https://www.phper.ninja/>
- <https://zenn.dev/tadsan>
- <https://qiita.com/tadsan>
- <https://github.com/bag2php>

WEB+DB PRESS総集編



Software Design 11月号



この記事は執筆時点で10月2～3日に開催を予定しているPHPカンファレンス2021と連動した短期集中連載の最終回です。つまり、この記事が掲載されたSoftware Design 2021年11月号をみなさんが手にとられるころにはすでに開催されています。カンファレンスで発表されたセッションはYouTubeチャンネル^{注1}にアーカイブが公開されているはずですので、興味のある方はぜひご覧ください。筆者はPHPカンファレンス2021で「配列、ジェネリクス、PHPで書けない型」と題したトークを発表します。

最終回では近年の動的言語の型検査の動向についての概観に触れたあと、PHPの動向について説明します。

動的型付き(statically typed)の言語と呼びます。事前にデータの種類を固定せずに処理することを動的型検査(dynamic type checking)と呼び、実行時にデータに応じた処理をすることが基本のプログラミング言語を動的プログラミング言語(dynamic programming language)と呼びます。

メジャーなものでは、C言語、Java、C++、C#、Go、Swift、Rust、Scala、Kotlin、Haskell、OCamlなどは静的型付きの言語です。また、Python、JavaScript、Ruby、シェルスクリプト、ClojureなどLisp系の言語、Smalltalk、そしてPHPは動的言語と呼ばれます。俗に実行ファイルを生成するものをコンパイル言語、ソースコードを直接実行するものをインタプリタ言語として

ツナパハに行きたかった2022



前回までのあらすじ

最後のPHP勉強会(2020年)

≡ PIXIV FANBOX



tadsan

クリエイターページ

投稿する

ダッシュボード

投稿管理

クリエイターページ管理

プラン管理

クリエイターID設定

支援者一覧

フォロワー一覧

おたより

支援金管理/振込

二次元コード発行

配列の型と向き合う

2020-02-26 PHP勉強会@東京

2020年2月27日 02:33

全体公開

arrayの型と向き合う #phpstudy

2020年最初のPHP勉強会@東京である第145回 PHP勉強会@東京で話してきました。話の経緯としては、この数日ざっくり書いている型について記事が一向に書き上げられなくてむしゃくし...



PHPカンファレンス沖縄2021

YouTube Premium JP 検索

▶ 4.44 ★★★★★ PHPCONF OKINAWA 2021

今日からできる安心型付け入門
Introduction of typing in PHP

2021-05-29 YouTube
PHPカンファレンス沖縄

Platinum Sponsor PHP 技術者認定機構 Toysub!

#phpcon_okinawa

PHPカンファレンス沖縄2021 Track A

2,107 回視聴 • 2021/05/29 にライブ配信

32 低評価 共有 オフライン 保存 ...

PHPカンファレンス沖縄
チャンネル登録者数 60人

チャンネル登録

Twitter feed:

- #phpcon_okinawa #track_a
今日も誰かのにちようび(おい... 14s
(でも「コミットの粒度」みたいな話
をして気分良く会話が終わった経験
がないかもな... 膝に矢を受けて
しまった...)
- #phpcon_okinawa #track_a
たつきち フリーランス時々CT... 2m
コミットメッセージは
github.com/streamich/git-... 使って書
くのが個人的にはおすすめです!
- #phpcon_okinawa
ボンちゃん@ビットラボWEB工... 3m
新たに知識を得る事は快感^ - ^
- #phpcon_okinawa #track_a
そーだい@初代ALF @soudai1025 5m
- Takuto Wada @t.wada
Replying to @hagevashi
Git のコミットコメントの規約として
は、Why の分量が多い時はコミット
のタイトル (1行目) には概要を書
き、空行を挟んで3行目以降に Why
を書いていくのがオススメです。

PHPカンファレンス2021

採択 2021/10/03 10:00～ Track2 Long session (60 mins)

配列、ジェネリクス、PHPで書けない型

PHP Conference Japan 2021

☆ 6



うさみけんた  [tadsan](#)

近年、PHPの機能強化により型宣言だけで安全に書ける範囲が広がっています。

その一方でPHPStanやPsalmといった静的解析ツールはPHPでは表現できない強力な型を提供することでコード品質向上の価値を高めることが可能です。

PhpStormはPHPプログラマに静的型の恩恵をもたらした一方、先述のツールと比べサポートする型について見劣りする点がありました。

ところが最近リリースされた待望の新バージョンである2021.2はこれまで静的解析ツールの専売特許だった型のいくつかをサポートされるようになり、型検査だけでなく入力補完などの恩恵を受けられるようになりました。そこで追加された型のひとつがジェネリクス（総称型）です。

本発表では、PHPの型についての基礎知識（[今日からできる安心型付け入門](#)）があることを前提として、[PHPにジェネリクスは入るのか？](#)の内容を軸に、PHPの型宣言では現時点で賄えない部分、特に配列の型およびジェネリクスの概念、class-string型の概念、そしてジェネリクスを実際に活用するためのテクニックを説明します。

Discord Channel: [#track2-4-php-type](#)

PHPerKaigi petit 8.1 (2021年)

要約：
型宣言arrayだけでは
不十分なので、もっと
詳しく書かねばならぬ

著者一覧を出力したい

```
class Book
{
    function printAuthors($fp): void {
        foreach ($this->authors as $author) {
            fwrite($fp, $author->getName());
        }
    }
}
```

Book::\$authors : array
中身が何かわからない！

Authorクラスの
メソッドを補完できない

りくつはわかった

ならぬと言われても
どう書けばいいのか
わからん

配列に型を付ける(実践篇)

Practice to type array



pixiv Inc.
USAMI Kenta

pixiv

配列は単一の型ではないことを認める

PHPマニュアルより引用

php

Downloads

Documentation

Get Involved

Help

php8.1

[Dutch PHP Conference 2022 - Call for Papers](#)

PHP マニュアル > 言語リファレンス > 型

今回の主題はこれではないので

change language: Japanese ▼

[Submit a Pull Request](#)

[Report a Bug](#)

配列

一旦忘れてください

PHP の配列は、実際には順番付けられたマップです。マップは型の一種で、*値*をキーに関連付けます。この型は、さまざまな使い道にあわせて最適化されます。配列としてだけでなく、リスト (ベクター)、ハッシュテーブル (マップの実装の一つ)、辞書、コレクション、スタック、キュー等として使用することが可能です。PHP の配列には他の PHP 配列を値として保持することができるため、非常に簡単にツリー構造を表現することが可能です。

これらのデータ構造に関する説明は本マニュアルの範囲を超えるので省略しますが、各々について、少なくとも一つは例を示します。この分野は広範囲にまたがり、さまざまな文献が存在します。より詳細な情報については、これらの文献を参照ください。

重要なのは
データの持ちかた

配列はひとつ！じゃない！！

list<int>

array<int,string>

- ユーザIDがならんだリスト [123, 456]
- ユーザIDと名前の対応表 [123 => '野比のび太', 234 => '源静香']
- ユーザを表す構造体 ['id' => 123, 'name' => '野比のび太']
- ユーザを表す構造体がならんだリスト
[
 ['id' => 123, 'name' => '野比のび太'],
 ['id' => 234, 'name' => '源静香'],
]

array{id:int, name:string}

list<array{id:int, name:string}>

配列はどこからくるの

arrayのふるさと

- PHPコードに書かれたリテラル
- HTTPリクエスト (`$_GET`, `$_POST`)
- デシリアライズ (JSON, PHPシリアライズ)・eval
 - `json_decode(file_get_contents(_DIR_ . '/config.json'))`
 - 外部APIリクエスト
- `PDOStatement::fetch()` 系メソッド

arrayリテラル

- 旧式の `array()` とPHP 5.4以降の `[]` がある
- `['a', 'b']` と `[0 => 'a', 1 => 'b']`
 - キーを省略すると 0 はじまりになる
- 重要: `array()` は関数ではない
 - `array('key' => 'value')` は書けるが
`hoge('key' => 'value')` はsyntax errorになる

デシリアライズ・eval

- 文字列に変換されたデータを解凍した結果
 - `$a = json_decode("[1, 2]", true);`
 - `$b = unserialize("a:2:{i:0;i:1;i:1;i:2;}");`
 - `$c = eval('return [1, 2];');`

PDOの場合

- PDO::fetch() に渡すオプションによって配列の形状が変わる
- SELECT id, name FROM users; のようなSQLを考える
 - PDO::FETCH_ASSOC → ['id' => '1', 'name' => '初音ミク']
 - PDO::FETCH_NUM → [0 => '1', 1 => '初音ミク']
 - PDO::FETCH_BOTH →
[0 => '1', 1 => '初音ミク', 'id' => '1', 'name' => '初音ミク']

PDOの場合 (fetchAll)

- さらにfetch(の結果が並ぶ)
- PDO::FETCH_ASSOC → [
 ['id' => '1', 'name' => '初音ミク'],
 ['id' => '2', 'name' => '鏡音リン'],
 ['id' => '3', 'name' => '鏡音レン'],
]

HTTPリクエスト(\$_GET, \$_POST)

- HTMLフォームからは実は配列も送れる
 - ?data[]=a&data[]=b
 - \$_GET['data'] = ['a', 'b']
 - ?user[name]=a&user[addr]=b
 - \$_GET['user'] = ['name' => 'a', 'addr' => 'b']
- 'user_name', 'user_addr' みたいなのを手で展開するより便利

なぜ配列に
型がつかないのか

(再掲)デシリアライズ・eval

- 文字列に変換されたデータを解凍した結果
 - `$a = json_decode("[1, 2]", true);`
 - `$b = unserialize("a:2:{i:0;i:1;i:1;i:2;}");`
 - `$c = eval('return [1, 2];');`

実際のデシリアライズ・eval

- 文字列に変換されたデータを解凍した結果
 - `$a = json_decode((string)$res->getBody(), true);`
 - `$b = unserialize($serialize);`
 - `$c = eval($code);`
- 変数に入っていたり、メソッドから動的に生成されたりする
 - = ソースコードを見てもわからない

型宣言にarrayとしか書かないと

- メソッドに何を渡せばいいのかわからない
- どんな形式の配列を渡しても動くという関数は決して多くない
 - 標準関数の `count()` のようなユーティリティが多め
- プロパティにも何が入ってるのかわからない
 - 従来のLaravelは `@var array` まみれだった
 - Laravel 9.xのコードベースでは配列の内部構造に型がついている

たとえばこういう関数

```
/** DBからユーザーを取得する */  
function findUsers(array $ids): array {}  
  
// こういうパラメータを渡されても困る  
$users = findUsers(['hoge' => 'fuga'])  
  
// $idsというパラメータから推測しろ？ うーんわからん
```

たとえばこういう関数

```
/** 検索する */  
function search(string $word, array $opt): array {}  
  
$result = search('検索したい単語');  
  
// 実装を確認しに行かないとわからない  
$result = search('検索したい単語', [?????]);
```

配列につける型

従来多用されていた記法

```
/**
 * @param User[] $users
 */
function printUsers(array $users): void {
    foreach ($users as $user) {
        echo $user->name, "\n";
    }
}
```

従来記法の欠点

- 配列インデックスの型を指定できない
- 以下の配列が区別できない
 - `[12 => new User(), 34 => new User()]`
 - `['太郎' => new User(), '花子' => new User()]`
 - `[new User('太郎'), new User('花子')]`

PSR-5式の配列記法(ジェネリクス)

- 配列インデックスの型を指定できる
- 以下の配列が区別できる
 - [12 => new User(), 34 => new User()]
 - ['太郎' => new User(), '花子' => new User()]
 - [new User('太郎'), new User('花子')]

array<int,User>

array<string,User>

list<User>

従来多用されていた記法

```
/**
 * @param User[] $users
 */
function printUsers(array $users): void {
    foreach ($users as $user) {
        echo $user->name, "\n";
    }
}
```

旧PSR-5式の記法

```
/**
 * @param array<User> $users
 */
function printUsers(array $users): void {
    foreach ($users as $user) {
        echo $user->name, "\n";
    }
}
```


変数リテラルや定数は @var を避ける

- 静的解析ツールの実装にもよるが、リテラルから構造を分析できる
 - PHPStanなどは構造を読み取ってくれる
- むやみに書くと情報量が落ちる